

Manual Técnico — Whispers in Letters

Proyecto: Adaptación a videojuego del libro *El Mundo de Sofía*
Motor: Unity (3D, third-person) | **Narrativa:** Ink (Inkle Studios) | **Input:** Unity Input System

Índice General

Cap	Contenido
1	Arquitectura General — Managers, dependencias, ciclo de vida
2	Sistema Narrativo — Ink, StoryManager, diálogo, retratos, prólogo
3	Sistema de Interacción — PlayerInteraction, InteractableObject, triggers
4	Escenas y Niveles — LevelManager, SpawnPoint, transiciones
5	Sistema de UI — Menú, carrusel, ajustes, HUD
6	Sistema de Persistencia — SaveSystem, GameStateSO, PlayerPrefs
7	Guía de Creación de Contenido — Pipelines, checklists, riesgos
8	Convenciones Técnicas — Código, Ink, buenas prácticas, deuda técnica

Documentación externa: `docs/README.md` — tabla de sistemas, roadmap.

Capítulo 1: Arquitectura General

Patrón **Singleton-based centralizado con ScriptableObjects como datos**. Managers persistentes (`DontDestroyOnLoad`) en `Game Manager.prefab`. Comunicación: referencias directas a singleton, eventos C# nativos, funciones externas vinculadas desde Ink.



Syntax error in text
mermaid version 11.15.0

Manager	Dependencias
<code>GameManager</code>	GameStateSO, SaveSystem
<code>LevelManager</code>	GameManager
<code>StoryManager</code>	GameManager, DialogueUIController, inkJSON

Manager	Dependencias
AudioManager	4 AudioSource, PlayerPrefs
PrologueManager	GameManager, StoryManager

Ciclo de inicio: GameManager.Awake() → carga SaveSystem → clearStateOnStart? → AudioManager carga volúmenes → StoryManager.InitializeStory() → LevelManager configura fade → PrologueManager se registra para sceneLoaded.

Ciclo de vida al cargar escena: SceneManager.sceneLoaded dispara → GameManager.UpdateUIVisibility(), StoryManager.RefreshUIReferences(), LevelManager.ResetFade(), PrologueManager.HandleSceneRoutine().

Comunicación: (1) Eventos C# nativos (DialogueTagProcessor → DialogueUIController, OnDialogueStateChanged, OnInteractableChanged). (2) Referencias directas a singleton (patrón dominante). (3) Funciones externas de Ink vinculadas por StoryManager. (4) Tags Ink como comandos (#scene: , #setflag: , etc.).

Riesgo principal: GameManager es referenciado por ~13 clases (God Object incipiente). StoryManager por ~10 clases. PlayerMovement acoplado a StoryManager.IsDialogueActive.

Capítulo 2: Sistema Narrativo

Arquitectura en 4 capas: Ink (.ink → .json) → StoryManager (motor Unity) → DialogueUIController (UI) → Trigger Narrativo (gameplay).

Archivos Ink

Archivo	Líneas	Knots	Propósito
Historia.ink	262	~20	Entry point, INCLUDEs, prólogo, finales
Prologo.ink	84	9	Prólogo jugable
Objects.ink	310	~45	Interacciones con objetos, NPCs, patos
Joseph_Arcade.ink	307	~15	Ruta Arcade
Joseph_Arcade_Camino2.ink	157	~8	Segunda ruta Arcade
Joseph_Biblioteca.ink	303	~15	Ruta Biblioteca
Joseph_Biblioteca_Camino2.ink	174	~8	Segunda ruta Biblioteca
Epilogos.ink	75	8	Cartas de filósofos

Todos incluidos vía INCLUDE desde Historia.ink, compilado a Historia.json.

Tags de Ink

Tag	Ejemplo	Acción
#sprite:	#sprite:sophia_happy	Cambia retrato + identifica personaje
#setflag:	#setflag:conocio_sophia	Activa flag en GameState

Tag	Ejemplo	Acción
#deleteflag:	#deleteflag:exploracion	Desactiva flag
#setvar:	#setvar:ruta:Epilogo1	Asigna variable narrativa
#incrementvar:	#incrementvar:contador:1	Incrementa variable numérica
#sonido:	#sonido:joseph_suspiro	Reproduce SFX
#scene:	#scene:Biblioteca	Cambia de escena
#fade_out	#fade_out	Fundido a negro al terminar diálogo
#small_image:	#small_image:img1,img2	Muestra imágenes (máx 3)
#big_image:	#big_image:Control_Arcade	Muestra imagen grande

StoryManager

Assets/Scripts/Narrative/Narrative Logic/StoryManager.cs. Inicializa `Ink.Runtime.Story` desde el JSON, vincula funciones externas (`GetFlag`, `GetVar`), gestiona ciclo de diálogo. Expone `OnDialogueStateChanged(bool)`.

API: `StartStory(knot)` → elige path en Ink, inicia UI. `AdvanceStory()` → avanza línea. `IsDialogueActive` → consulta estado.

DialogueUIController

`DialogueUIController.cs`. Controla: texto typewriter, botones de opciones, retratos animados, voces, SFX, imágenes. Trabaja con `DialogueTagProcessor` que interpreta tags y redirige a GameManager/LevelManager/AudioManager.

PrologueManager

`Assets/Scripts/Core/PrologueManager.cs`. Orquesta prólogo a través de 3 escenas (Parque → Arcade/Biblioteca → Parque final). Evalúa flags y dispara knots al cargar escena.

Flag	Cuándo se setea
prologue_arcade_visited	Al llegar al Arcade
prologue_arcade_item_collected	Al recoger objeto en Arcade
prologue_library_visited	Al llegar a Biblioteca
prologue_library_item_collected	Al recoger objeto en Biblioteca
prologue_completed	Seteado por Ink al terminar
prologue_final_seen	Diálogo final visto

Triggers narrativos (cómo se inicia un diálogo)

Trigger	Componente	Modo
Objeto interactuable	<code>InteractableObject</code> + <code>InteractableData</code> SO	Al presionar botón

Trigger	Componente	Modo
Objeto condicional	<code>AdvancedInteractableObject</code> + <code>AdvancedInteractableData</code>	Condiciones AND
Objeto de prólogo	<code>PrologueItemInteractable</code>	Al presionar botón, auto-desactiva
Puerta/transición	<code>SceneTransitionTrigger</code>	Trigger automático + opcional confirmación
Automático al cargar	<code>PrologueManager</code>	Evalúa flags en <code>sceneLoaded</code>

Persistencia narrativa

`GameStateSO` almacena `List<string> flags`, `List<StoryVariable> variables`. El auto-save ocurre en `LevelManager.ChangeScene()` antes de cargar la nueva escena.

ScriptableObjects narrativos

Asset	Tipo	Propósito
<code>Estado.asset</code>	<code>GameStateSO</code>	Estado global del juego
<code>Images.asset</code>	<code>NarrativeImageDatabase</code>	Mapea IDs de imagen → Sprite
<code>Portrait Data Base.asset</code>	<code>CharacterPortraitData</code>	Mapea IDs de sprite → Animation State
<code>Cartas.asset</code>	<code>PhilosopherCardDatabase</code>	Datos de cartas de filósofos
<code>Patos DB.asset</code>	<code>CollectableDuckDatabase</code>	Datos de patos coleccionables

Mapa narrativo (árbol de decisión)

Arcade → Schopenhauer/Hegel. Biblioteca → Estoicos/Nietzsche. Cada ruta tiene 2 caminos con 2 finales cada uno (aceptación/reproche) = 4 finales totales + 1 epílogo por filósofo. ~45 objetos interactivables, 4 NPCs, 6 patos coleccionables.

Capítulo 3: Sistema de Interacción

Arquitectura en 4 capas: Collider Trigger → PlayerInteraction (IInteractable) → Implementaciones → Datos+Narrativa.

IInteractable

Interfaz raíz en `Assets/Scripts/Narrative/Interactions/Logic/IInteractable.cs`. Métodos: `void Interact()`, `string GetInteractionName()`.

PlayerInteraction

`Assets/Scripts/Player data/PlayerInteraction.cs`. Singleton en `Player.prefab`. Detecta `IInteractable` via `OnTriggerEnter/Exit`. Enlaza input (Tecla E / Gamepad South). Si hay diálogo activo, llama `AdvanceStory()` en lugar de `Interact()`. Expone evento `OnInteractableChanged(Action<IInteractable>)`.

Implementaciones

Clase	Archivo	SO de datos	Propósito
<code>InteractableObject</code>	<code>Interactions/SO/SO</code> <code>Logic/</code>	<code>InteractableData</code>	Objeto genérico (narrativo o coleccionable)
<code>AdvancedInteractableObject</code>	<code>Interactions/SO/Joseph/</code>	<code>AdvancedInteractableData</code>	Objeto condicional con lista de <code>InteractionEntry</code>
<code>PrologueItemInteractable</code>	<code>Interactions/Logic/</code>	Ninguno (campos directos)	Objeto de prólogo autocontenido
<code>SceneTransitionTrigger</code>	<code>Interactions/Logic/</code>	Ninguno	Transición de escenas (NO implementa <code>Interactable</code>)
<code>InteractableTable</code>	<code>Interactions/Logic/</code>	Ninguno	Cartas de filósofos (epílogo)
<code>FinalReflectionInteractable</code>	<code>Narrative Logic/</code>	<code>PhilosopherCardDatabase</code>	Reflexión final (4 etapas)

InteractableData SO

Creado desde `Create > Narrative > Interactable`. Campos: `interactionName`, `inkKnot`, `isCollectable`, `flagToSetOnCollect`, `variableToIncrementOnCollect`, `incrementAmount`.

AdvancedInteractableData SO

Creado desde `Create > Narrative > Advanced Interactable`. Lista de `InteractionEntry` evaluadas en orden (primera que cumple gana). Cada entrada tiene: `inkKnot`, `conditions` (AND lógica). Tipos de condición: `Ninguna`, `RequerirFlag`, `RequerirVariable`.

Prefabs Base

Prefab	Componentes de interacción
<code>Player.prefab</code>	<code>PlayerInteraction</code> , <code>InteractUI</code>
<code>Cambio de escena.prefab</code>	<code>SceneTransitionTrigger</code>
<code>IDEL JOSEPHIdle.prefab</code>	<code>AdvancedInteractableObject</code>
<code>JOSEPH.prefab</code>	<code>AdvancedInteractableObject</code>
<code>Gamepad_Classic.prefab</code>	<code>InteractableObject</code>

Dependencias

Todos los interactables dependen de `GameManager.Instance` (flags/variables), `StoryManager.Instance` (StartStory, IsDialogueActive) y opcionalmente `LevelManager.Instance` (ChangeScene).

Capítulo 4: Escenas y Niveles

Escenas registradas

Índice	Escena	Propósito
0	<code>Menu.unity</code>	Menú principal, carrusel, ajustes, créditos
1	<code>Arcade.unity</code>	Ruta Schopenhauer/Hegel
2	<code>Biblioteca.unity</code>	Ruta Estoicos/Nietzsche
3	<code>Cuarto.unity</code>	Reflexión final, epílogo
4	<code>Parque.unity</code>	Prólogo, encuentro social

Flujo: Menu → Parque → (Arcade ↔ Parque ↔ Biblioteca) → Cuarto → Menu.

LevelManager

Singleton persistente en `Game Manager.prefab`. API: `ChangeScene(sceneName)` (fade + carga + spawn completo), `FadeToBlack()`, `FadeToClear()`.

SpawnPoints

Cada escena contiene SpawnPoints con `fromSceneName`. PlayerManager busca coincidencia (case-insensitive) y teletransporta al jugador.

Fuentes de transición

Origen	Mecanismo
CarruselNiveles	<code>GameManager.RequestLoadLevel("Parque")</code>
SceneTransitionTrigger	<code>LevelManager.ChangeScene(destino)</code>
Botón "Salir al menú"	<code>LevelManager.ChangeScene("Menu")</code>
Tag <code>#scene:nombre</code> en Ink	<code>DialogueTagProcessor</code> → <code>LevelManager.ChangeScene()</code>
GameSummaryUI	<code>LevelManager.ChangeScene("Menu")</code>

SceneTransitionTrigger

Tres modos: (1) **Directo** — `ChangeScene()` inmediato. (2) **Con flag requerido** — evalúa requiredFlag + pila de condiciones, muestra fallback si no cumple. (3) **Con confirmación** — dispara knot de Ink que usa `#scene:` para la transición.

SceneMusicStarter

Cada escena debe tener un GameObject con este componente. Reproduce música y ambiente al iniciar la escena.

Dependencias por escena

Escena	Managers	Componentes de escena
Menu	GameManager, AudioManager	MenuInicio, CarruselNiveles, UIAjustes
Parque	Todos	Player, SceneTransitionTrigger(xN), SceneMusicStarter, SpawnPoint(xN)
Arcade	Todos	Player, SceneTransitionTrigger, SpawnPoint(x2), AdvancedInteractableObject (Joseph)
Biblioteca	Todos	Player, SceneTransitionTrigger, SpawnPoint(x2), AdvancedInteractableObject (Joseph)
Cuarto	Todos	Player, SceneMusicStarter, SpawnPoint, InteractableTable(x2), FinalReflectionInteractable

Capítulo 5: Sistema de UI

Tres subsistemas independientes: **Menu UI** (menú principal), **Dialogue UI** (diálogos y cartas, ver Cap. 2), **Interaction UI** (indicador de interacción, ver Cap. 3). No hay UIManager centralizado.

Canvases

Canvas	Render Mode	Orden	Propósito
Menu Canvas	ScreenSpace Overlay	0	Paneles del menú
Settings Canvas	ScreenSpace Overlay	1	Ajustes
Dialogue Canvas	ScreenSpace Overlay	2	Diálogos, opciones, retratos
Card Canvas	ScreenSpace Overlay	3	Cartas de filósofos
Interaction Canvas	WorldSpace	N/A	Billboard de interacción
Fade Canvas	ScreenSpace Overlay	10	Fundido a negro
Summary Canvas	ScreenSpace Overlay	4	Resumen final

GameManager y HUD

GameManager controla visibilidad de `hudObjects[]`. Tecla Q abre/cierra panel de ajustes (pausa Time.timeScale).

Menú Principal

`MenuInicio.cs`: Jugar → CarruselNiveles, Créditos, Ajustes, Salir. `CarruselNiveles.cs`: navegación A/D, muestra hasta 4 estrellas por final desbloqueado vía PlayerPrefs. `UIAjustes.cs`: 5 sliders (Música, SFX, UI, Ambiente, Master) vinculados a AudioManager.

Clases de UI

Clase	Archivo	Responsabilidad
MenuInicio	Menu/MenuInicio.cs	Navegación del menú principal
CarruselNiveles	Menu/CarruselNiveles.cs	Selector de nivel con estrellas
UIAjustes	Menu/UIAjustes.cs	Panel de configuración de volumen
BotonSalirMenu	Menu/BotonSalirMenu.cs	Auto-save + ChangeScene("Menu")
TutorialCarrusel	Menu/TutorialCarrusel.cs	Tutorial interactivo
GameSummaryUI	Core/GameSummaryUI.cs	Pantalla final del juego
GameSummaryManager	Core/GameSummaryManager.cs	Lógica del resumen final

Capítulo 6: Sistema de Persistencia

Arquitectura

```

GameManager (API pública)
  → GameStateSO (estado en runtime)
  → GameSaveData (POCO serializable)
  → SaveSystem (static, I/O disco/PlayerPrefs)

```

Flujo Save/Load

```

SaveGame(): GameStateSO → GameSaveData → SaveSystem.Save() → save.json
LoadGame(): SaveSystem.Load() → GameSaveData → GameStateSO.LoadFrom(data)

```

Auto-save en cada `LevelManager.ChangeScene()` (antes de cargar la nueva escena). Auto-save en coleccionables tras recoger.

GameStateSO

`Assets/Scripts/Player data/GameStateSO.cs`. ScriptableObject creado desde `Core/Game State`. Almacena: `currentSceneName`, `previousSceneName`, `playerPosition`, `playerRotation`, `flags` (List), `variables` (List clave-valor).

SaveSystem

Clase estática en `Assets/Scripts/Core/SaveSystem.cs`. Solo GameManager lo invoca. Escribe/lee `save.json` en `Application.persistentDataPath` (o PlayerPrefs en WebGL).

Datos persistidos

Dato	Origen	Destino
Flags narrativos	GameStateSO	save.json
Variables narrativas	GameStateSO	save.json

Dato	Origen	Destino
Escena actual	GameStateSO	save.json
Posición del jugador	GameStateSO	save.json
Volúmenes de audio	AudioManager	PlayerPrefs
Finales desbloqueados	GameSummaryManager	PlayerPrefs

PlayerPrefs Keys

PlayerPrefsKeys.cs — constantes centralizadas. Volúmenes: MusicVolume, SFXVolume, UIVolume, AmbientVolume, MasterVolume. Finales: EndingUnlocked_Schopenhauer, EndingUnlocked_Hegel, EndingUnlocked_Estoicos, EndingUnlocked_Nietzsche.

Eventos de guardado

OnDialogueStateChanged(false) → GameSummaryManager evalúa si mostrar resumen; AdvancedInteractableObject evalúa si desaparecer.

Capítulo 7: Guía de Creación de Contenido

7.1 Visión General

Este capítulo unifica pipelines, checklists y riesgos de los capítulos anteriores. Es la referencia única para creación de nuevo contenido.

7.2 Cómo crear un nuevo objeto interactuable

1. Crear InteractableData SO (Assets/Create/Narrative/Interactable) con interactionName, inkKnot
2. GameObject con Collider (IsTrigger=true) + InteractableObject component
3. Arrastrar SO a data
4. Opcional: requiredFlag, fallbackKnot, autoTriggerOnEnter

Coleccionable: marcar isCollectable, configurar flagToSetOnCollect. Se desactiva automáticamente si el flag ya existe al cargar.

7.3 Cómo crear una secuencia interactiva (puzzle)

1. Crear AdvancedInteractableData SO (Assets/Create/Narrative/Advanced Interactable)
2. Agregar InteractionEntry por cada knot posible con condiciones AND
3. GameObject con Collider + AdvancedInteractableObject
4. Opcional: visibilityConditions (ocultar si no se cumplen), disappearAfterDialogue

7.4 Cómo crear una nueva interacción narrativa

1. Escribir knot en archivo .ink (o crear nuevo archivo + INCLUDE en Historia.ink)
2. Usar tags #sprite:, #setflag:, #sonido:, etc. según necesidad
3. Compilar .json desde Inky (Ctrl+S)
4. En Unity, asignar el knot desde el Inspector del interactuable correspondiente
5. Verificar en consola: [StoryManager] StartStory llamado con knot: '...'

7.5 Cómo crear un nuevo nivel

1. Crear escena en `Assets/Scenes/`
2. Registrar en Build Settings (verificar índice)
3. Agregar `SceneMusicStarter`, `SpawnPoints`, Player prefab
4. Configurar `SceneTransitionTrigger` para entrada/salida
5. Si es parte del prólogo: agregar handler en `PrologueManager.HandleSceneRoutine()`
6. Agregar interactivables según la narrativa

7.6 Cómo crear un interactuable condicional

1. Crear `AdvancedInteractableData` SO
2. Agregar `InteractionEntry` por cada knot, en orden de prioridad
3. Configurar condiciones: `RequerirFlag` o `RequerirVariable`
4. Asignar SO al `AdvancedInteractableObject` en escena
5. Probar cada rama condicional

7.7 Cómo reutilizar prefabs

- Prefab base = configuración mínima funcional (Collider trigger + script)
- Variantes: arrastrar a escena y asignar SO específico, o crear Prefab Variant
- Para Joseph/NPC condicional: usar prefab existente, cambiar `AdvancedInteractableData` SO

7.8 Cómo registrar un nuevo sistema (manager)

1. Crear MonoBehaviour con patrón singleton (`Instance + DontDestroyOnLoad`)
2. Agregar al `Game Manager.prefab`
3. Si necesita acceso global: exponer API pública y eventos C# si aplica
4. Si necesita reaccionar a cambios de escena: suscribirse a `SceneManager.sceneLoaded`
5. Si necesita reaccionar al diálogo: suscribirse a `StoryManager.OnDialogueStateChanged`

7.9 Cómo conectar gameplay y narrativa

Unity → Ink: `StoryManager.Instance.StartStory("knot_name")` desde cualquier `IInteractable`. El knot debe existir en archivo .ink compilado.

Ink → Unity: Tags en el script Ink (`#setflag:`, `#scene:`, `#sonido:`, etc.) son procesados por `DialogueTagProcessor` y redirigidos al manager correspondiente.

Transición desde Ink: `#scene:NombreEscena` en el knot. El `SceneTransitionTrigger` con `confirmationKnot` dispara el diálogo, y el tag completa la transición.

7.10 Checklist General de Creación de Contenido

Pre-creación

- [] El knot tiene nombre único en todo el proyecto
- [] El archivo .ink está INCLUDE en `Historia.ink`
- [] El .json fue recompilado desde Inky
- [] Tags con sintaxis correcta (`#tag:valor`, snake_case para flags)
- [] `#sprite:` existe en `CharacterPortraitData`
- [] `#sonido:` existe en `DialogueUIController.dialogueSounds`

- [] `#small_image:` / `#big_image:` existe en `NarrativeImageDatabase`

Implementación

- [] Collider con `IsTrigger=true` en el `GameObject`
- [] `InteractableObject/AdvancedInteractableObject` con SO asignado
- [] `inkKnot` coincide exactamente con `knot` en `Ink`
- [] Si usa `requiredFlag`: el flag se setea en el momento correcto
- [] Si usa condiciones: no hay conflictos ni bucles infinitos
- [] `flagToSetOnCollect` único en todo el proyecto (coleccionables)
- [] `destinationSceneName` coincide con `Build Settings` (transiciones)

Persistencia

- [] Flags/variables se persisten al cambiar de escena
- [] Objetos ya recogidos no aparecen al recargar escena
- [] auto-save se llama antes de transicionar

Integración

- [] El diálogo inicia correctamente al interactuar
- [] Opciones llevan al `knot` correcto
- [] El prólogo fluye correctamente entre escenas
- [] Al terminar el diálogo, el panel se oculta y el personaje puede moverse
- [] Los managers persistentes no se duplican al recargar Menu

7.11 Riesgos Frecuentes

#	Riesgo	Mitigación
1	Knot mal escrito en C# o SO	Usar constantes de strings. Verificar coincidencia exacta
2	Tag malformado (<code>#setflag:flag name</code>)	Los espacios internos no se corrigen. Usar <code>snake_case</code>
3	INCLUDE faltante	Verificar que el <code>.ink</code> está en <code>Historia.ink</code>
4	No recompilar .json	Configurar auto-compilar en <code>Inky</code>
5	GameState no persiste	Verificar que transiciones pasan por <code>LevelManager.ChangeScene()</code>
6	Diálogo bloqueado	Verificar <code>!IsDialogueActive</code> antes de <code>StartStory</code>
7	Collider mal posicionado	Usar Gizmos en el Editor
8	Condiciones en conflicto (Advanced)	Revisar orden de entradas. Usar condiciones excluyentes
9	confirmationKnot sin tag <code>#scene:</code>	El script <code>Ink</code> debe incluir <code>#scene:NombreEscena</code>
10	SpawnPoint sin <code>fromSceneName</code>	Cada escena debe tener <code>SpawnPoint</code> para cada origen
11	Fade atascado en negro	<code>LevelManager.OnSceneLoaded</code> resetea el fade
12	Managers duplicados	Usan <code>DontDestroyOnLoad</code> + singleton check en <code>Awake</code>

7.12 Referencia rápida de archivos

Archivo	Ruta
GameManager.cs	Assets/Scripts/Core/
StoryManager.cs	Assets/Scripts/Narrative/Narrative Logic/
DialogueUIController.cs	Assets/Scripts/Narrative/Narrative Logic/
DialogueTagProcessor.cs	Assets/Scripts/Narrative/Narrative Logic/
LevelManager.cs	Assets/Scripts/Menu/
SaveSystem.cs	Assets/Scripts/Core/
GameStateSO.cs	Assets/Scripts/Player data/
PlayerInteraction.cs	Assets/Scripts/Player data/
InteractableObject.cs	Assets/Scripts/Narrative/Interactions/SO/SO Logic/
AdvancedInteractableObject.cs	Assets/Scripts/Narrative/Interactions/SO/Joseph/
PrologueManager.cs	Assets/Scripts/Core/
PrologueItemInteractable.cs	Assets/Scripts/Narrative/Interactions/Logic/
SceneTransitionTrigger.cs	Assets/Scripts/Narrative/Interactions/Logic/
AudioManager.cs	Assets/Scripts/Audio/
SceneMusicStarter.cs	Assets/Scripts/Audio/
MenuInicio.cs	Assets/Scripts/Menu/
CarruselNiveles.cs	Assets/Scripts/Menu/
GameManager.prefab	Assets/Prefabs/
Player.prefab	Assets/Prefabs/
Historia.json	Assets/Scripts/Narrative/Ink/

Capítulo 8: Convenciones Técnicas y Reglas Arquitectónicas

8.1 Convenciones de Código

- **Lenguaje:** C# en Unity, scripts Ink para narrativa
- **Nombres:** PascalCase para clases/métodos públicos, camelCase para params/vars privadas, `_camelCase` para campos serializados
- **Scripts:** un archivo por clase, nombre = clase
- **Namespaces:** no se usan (no hay colisiones significativas)
- **Eventos:** `Action<T>` para eventos simples, `event EventHandler<T>` para eventos estándar. Sufijo `Changed` para notificaciones de estado

- **Archivos Ink:** archivo raíz `Historia.ink` con INCLUDEs. Un archivo por ruta narrativa. `snake_case` para nombres de knot
- **Tags Ink:** `#tag:valor` sin espacios. `#setflag:` usa `snake_case`
- **SO:** usar `CreateAssetMenu` con rutas tipo `Narrative/Interactable`

8.2 Estructura de Carpetas

```
Assets/Scripts/
  Core/                - GameManager, SaveSystem, GameStateSO, GameSummaryManager, PrologueManager
  Player data/         - PlayerManager, PlayerMovement, PlayerInteraction, GameStateSO
  Audio/               - AudioManager, AudioEvent, SceneMusicStarter
  Menu/                - LevelManager, MenuInicio, CarruselNiveles, UIAjustes, BotonSalirMenu
  Narrative/
    Narrative Logic/   - StoryManager, DialogueUIController, DialogueTagProcessor, CardPanelController
    Ink/               - Historia.ink, Historia.json
    Interactions/
      Logic/           - IInteractable, InteractableData, SceneTransitionTrigger, PrologueItemInteractable
      SO/SO Logic/     - InteractableObject, NarrativeImageDatabase, CharacterPortraitData
      SO/Joseph/       - AdvancedInteractableObject
  Prefabs/             - Game Manager.prefab, Player.prefab
  Scenes/              - Menu, Parque, Arcade, Biblioteca, Cuarto
```

8.3 Reglas de Dependencias

- Solo `GameManager` invoca `SaveSystem`
- `PlayerMovement` no debe llamar a managers directamente (actualmente rompe esta regla)
- Todos los interactables se comunican solo via `IInteractable` y sus SO
- La UI de diálogo depende de `StoryManager`, no de `GameManager`
- `AudioManager` solo usa `PlayerPrefs`, no `GameStateSO`
- Nuevos managers: agregar a `Game Manager.prefab` + patrón singleton + `DontDestroyOnLoad`
- Nuevos interactables: implementar `IInteractable`, no heredar

8.4 Buenas Prácticas

1. **Usar SO, no datos hardcoded** — permiten reutilizar lógica con diferentes datos
2. **Extraer PulseAnimation** a hijo del interactuable (desacoplado del script principal)
3. **Usar Prefab Variants** en lugar de modificar prefabs base directamente
4. **Constantes para flags y knots** — strings literales son frágiles
5. **Probar todas las ramas condicionales** — cada `InteractionEntry` debe probarse
6. **Eventos para comunicación entre sistemas** en lugar de referencias directas (futuro)
7. **Mantener interacciónName descriptivo** (se usa en logs de depuración)

8.5 Flujo Recomendado de Git

1. Crear rama desde `develop`: `feature/nueva-interaccion`
2. Implementar contenido Ink + configuración Unity
3. Probar en Unity Play mode
4. Commit: mensaje descriptivo con prefijo (`[Narrativa]`, `[Interacción]`, `[UI]`)
5. Push + PR a `develop` con descripción de qué se probó

8.6 Prácticas Seguras para Modificar Sistemas

1. **StoryManager:** modificar solo para cambiar lógica de BindExternalFunction o ciclo de diálogo. No tocar si solo se agrega contenido Ink
2. **GameManager:** punto más sensible. Cualquier cambio requiere probar: inicio de partida, continuación, cambio de escena, guardado/carga
3. **LevelManager:** cambios en transiciones afectan todas las escenas. Probar fade, spawn, auto-save
4. **AdvancedInteractableObject:** cambios en condiciones requieren verificar que no rompen la lógica de Joseph

8.7 Reglas para Nuevos Desarrolladores

1. Leer este manual (Cap. 1-4 para entender el sistema, Cap. 7 para crear contenido)
2. Para agregar narrativa: solo tocar archivos .ink (no código C#)
3. Para agregar objetos: seguir pipeline del §7.2
4. No modificar managers sin entender el flujo completo
5. Probar siempre: partida nueva → interactuar → cambiar escena → guardar → cargar
6. Verificar logs en consola ante cualquier comportamiento extraño

8.8 Deuda Técnica Identificada

ID	Deuda	Impacto	Mitigación
DT1	PlayerMovement acoplado a StoryManager.IsDialogueActive	Medio	Migrar a evento OnDialogueStateChanged
DT2	GameManager como God Object (~13 dependencias)	Alto	Introducir event channels vía SO
DT3	StoryManager como segundo hub (~10 dependencias)	Alto	Event channels
DT4	Lógica de spawn duplicada en LevelManager y PlayerManager	Bajo	Unificar en un solo servicio
DT5	GameObject.Find() para localizar jugador	Bajo	Sistema de referencias por escena
DT6	PrologueItemInteractable con su propio sistema de pulso (antes de extraer PulseAnimation)	Bajo	Ya migrado
DT7	Falta de tests automatizados	Medio	Agregar tests de integración para flujos críticos
DT8	No hay versionado de save.json	Medio	Agregar schema version
DT9	InteractUI legacy coexistiendo con InteractionIndicator	Bajo	Deprecar InteractUI
DT10	Nombres de archivos inconsistentes (espacios vs sin espacios)	Bajo	Estandarizar

8.9 Áreas Frágiles

Área	Riesgo	Por qué es frágil
<code>LevelManager.ChangeScene()</code>	Alto	Orquesta fade, auto-save, carga asíncrona y spawn. Cualquier error deja el juego en estado inconsistente

Área	Riesgo	Por qué es frágil
<code>StoryManager.StartStory()</code>	Alto	Depende de inkJSON compilado, BindExternalFunction, DialogueUIController y GameState
<code>AdvancedInteractableObject</code> condiciones	Medio	La evaluación en orden es correcta pero frágil si se agregan nuevas condiciones sin revisar las existentes
<code>GameManager.RequestLoadLevel()</code>	Alto	Decide entre partida nueva, continuación o reinicio. Error aquí = bucle de carga
<code>PrologueManager.HandleSceneRoutine()</code>	Medio	Depende de flags y nombres de escena hardcodeados. Nueva escena de prólogo = nuevo handler

8.10 Mejoras Futuras Recomendadas

1. **Bus de eventos desacoplado** (ScriptableObject event channels) para reducir acoplamiento a GameManager/StoryManager
2. **Eliminar duplicación de spawn** entre LevelManager y PlayerManager
3. **Migrar GameObject.Find()** a SceneReferences por escena
4. **Migrar consultas directas a singleton** (PlayerMovement → StoryManager) a suscripciones a eventos
5. **Agregar tests de integración** para flujos críticos (inicio → interactuar → transición → guardar → cargar)
6. **Versionado de save** para evitar corrupción entre versiones

Fin del manual. Documentación externa: `docs/README.md` — *tabla de sistemas y roadmap.*